

Modalités

- Durée : plusieurs semaines en mode « mini projet »
- Technologies :
 - BackEnd : API **Spring Boot**. Stockage **H2**
 - FrontEnd : Projet **Angular 21.x**
- Travail individuel.
- Lors de l'entretien suivant le rendu, vous devez être capable d'expliquer l'intégralité du code et de répondre aux questions du correcteur.
- Vous devrez travailler cette évaluation **en dehors de vos créneaux à l'école. Les créneaux PI ou de préparation au jury ne sont pas faits pour cela**. Vous avez plusieurs semaines et week-ends pour travailler dessus.
- Recherches sur internet autorisées.
- Utilisation d'IA interdite.

Contexte

Vous devez réaliser une application n-tiers **sur le sujet de votre choix** en réalisant une **API Spring** en tant que back-end et un front **Angular** s'appuyant sur cette API pour les données.

Afin de valider les compétences et d'éviter les rendus trop légers, les indications suivantes devront être appliquées :

Restriction et obligations back-end

Architecture

- Vous architecturerez le projets back de façon logique en repartissant vos classes dans des packages pertinents
- Les classes de test doivent être dans un dossier source à part (réservé à celles-ci) dupliquant la structure des packages du dossier source principal.
- Si la notion d'utilisateur ne sert que pour la sécurité, le modèle, les repositories, services et endpoints liés à celui-ci seront dans un package à part pour ne pas « polluer » les packages fonctionnels.

Données (Bdd et modèle)

- Votre modèle de données doit comprendre au moins 3 classes avec au minimum une relation **@OneToMany** ou **@ManyToMany** entre deux de ces classes.
- De l'héritage peut être présent ou non dans ce modèle, mais s'il est souhaitable vu la nature de celui-ci, il doit être mis en place pour factoriser le code.
- Les annotations sur les colonnes doivent mettre en place des validations minimales sur les données en termes de présence obligatoire et d'unicité.
- Vous devez gérer, par annotation ou via le code les opérations CRUD sur les éléments liés (ex, par rapport au TP fil rouge : comment sont gérées les lunes lors de l'effacement de leur planète?)
- Les données seront stockées dans une base h2 au format fichier (pas de base en RAM sauf, éventuellement, pour les tests).

- Vous devrez intégrer au projet la console H2 afin de pouvoir vérifier le contenu de cette base via l'interface en client léger.
- Vous devrez fournir un script SQL permettant de créer le profil administrateur demandé dans la partie « Sécurité » ainsi que fournir son mot de passe non hashé (via un fichier txt ou en commentaire dans le fichier SQL).

Services

- Vous devrez mettre en place quelques règles métier afin que vos services ne se contentent pas d'appeler les repositories sans aucun test ni contrôle des données.

API

- Votre API respectera les bonnes pratiques et standard W3C, API REST et SPRING quand aux urls et méthodes HTTP. Vous devrez fournir, à minima, les méthodes de création, lecture, modification (totale ou partielle), suppression des entités du modèle ainsi que des endpoints permettant d'associer ou dissocier deux entités liées.
- Votre API doit utiliser le pattern DTO et ne pas exposer, via les endpoints, les entités du modèle.
- Toutes les exceptions éventuellement levées par vos endpoints devront être capturées et renvoyées au client dans un format identique.
- Les données en entrée des endpoints devront être vérifiées lorsque c'est possible, via le framework de validation de Spring, afin d'éviter les appels Bdd sur des données que l'on sait déjà incomplètes ou non pertinentes.
- Votre API devra être documentée via Swagger. Cette documentation devra faire apparaître les endpoints dont l'accès est limité à l'administrateur.

Configuration Spring

- Vous définirez au moins deux profils : un pour le développement (par défaut) et un pour les tests. Vous pouvez aussi définir un profil de production basé sur mySql, mais ce n'est pas obligatoire.

Logging

- Vous définirez deux sorties possibles pour le logging :
 - Une sortie console
 - Une sortie vers des fichiers se renouvelant toutes les 24h ou s'ils dépassent les 100Ko. Les anciens fichiers seront conservés dans un sous dossier du répertoire des logs.
- En profil dev, la sortie se fera exclusivement vers les fichiers au niveau WARNING ou plus, sauf pour les controllers qui traceront aussi en console et au niveau INFO.
- En profil test, tout est tracé vers la console au niveau DEBUG.
- Vous définirez au moins les traces suivantes **pour les controllers** :
 - Un DEBUG à chaque entrée dans un endpoint traçant l'url et la méthode HTTP à laquelle ce endpoint répond.
 - Un INFO quand le but de la méthode est atteint (on ressort de celle-ci avec un statut OK, CREATED, ou NO_CONTENT)
 - Une ERROR quand une exception est catchée.
- Vous définirez au moins les traces suivantes **pour les services** :
 - Un DEBUG à chaque entrée dans une methode du service traçant le nom de celle-ci.

Sécurité

- Vous devrez mettre en place un filtrage CORS pour n'accepter que les connections du client Angular.
- Parmi les endpoints de l'API, seuls ceux dédiés à la lecture seront « tout publics ». Les autres ne seront réservés qu'à un profil administrateur via un JWT.

Tests

- Votre API doit avoir un taux de couverture de test supérieur à 80 % au moins sur les **controllers** et les **services**. Ces tests doivent être pertinents et couvrir tous les cas d'usage, y compris les cas limites tels que :
 - des vérifications DTO entrants non validées
 - des tentatives d'accès via une autre origine que le client Angular
 - des tentatives d'accès aux méthodes CUD par un client non identifié comme administrateur
- Vous devez aussi tester les **repositories**.

Restriction et obligations front-end

- Vous architecturerez le projet front de façon logique en choisissant l'un des patterns fonctionnel→technique ou technique→fonctionnel tant que la logique des différentes couches/entités est respectée.
- Seuls les fichiers créés initialement par Angular doivent être présents dans le dossier **src/app**. Tous ceux que vous ajouterez doivent être mis dans des sous dossiers.
- Des routes devront être mises en place et notamment, un basculement vers une page « 404 not found » en cas d'URL ne correspondant à aucune route.
- Le front s'appuiera sur les données de l'API Spring fournie et uniquement sur celles-ci.
- Un système de connexion/déconnexion permettra à l'utilisateur de prendre le rôle administrateur ou de simple visiteur.
- La gestion des données passera par des services en charge de centraliser les accès et de fournir un « cache » de celles-ci pour tous les composants d'affichage.
- Les limitations dues au profil seront, **à minima**, gérées par des **guards** sur les routes correspondantes aux opérations CUD sur les données, au mieux en y associant des invisibilités de boutons ou liens si l'utilisateur n'est pas connecté.
- Les opérations de création et mise à jours devront passer par des formulaires de type **ReactiveForms** avec des **validators** correspondants aux restrictions de l'API.
- **Toutes les variables de votre code TypeScript devront être typées. AUCUN TYPE Any** sauf si explicitement renvoyé par une librairie externe utilisée.

Livrables

Vous rendrez votre travail à la fin du temps imparti sur le devoir Teams.

Livable Front

Le livrable de la partie front de votre projet devra contenir au moins un zip du dossier « src » (**UNIQUEMENT LE DOSSIER SRC**) de votre projet Angular. Ce ZIP devra IMPÉRATIVEMENT être nommé comme suit :

S4IL_FRONT_202606_<votreNom>_<votrePrenom>.zip

(en changeant, bien entendu, les valeurs <votreNom> et <votrePrenom> par les valeurs correspondantes).

Si vous avez ajouté des bibliothèques externes à votre projet (via un « ng add xxx ») ou si vous avez des éléments supplémentaires à fournir à votre correcteur front, vous pouvez ajouter à votre rendu un fichier d'explication ou de précisions qui sera nommé :

S4IL_FRONT_202606_<votreNom>_<votrePrenom>_README.txt

Livable Back

Le livrable de la partie back de votre projet consistera en un export de votre projet via STS :

- ☐ Sélectionnez le projet « Click Droit→Export...→General→Archive File »
- ☐ Vérifiez que votre projet (et uniquement lui) est bien coché.
- ☐ Vous déposerez cette archive dans Teams. Ce ZIP devra IMPÉRATIVEMENT être nommé comme suit :

S4IL_BACK_202606_<votreNom>_<votrePrenom>.zip

(en changeant, bien entendu, les valeurs <votreNom> et <votrePrenom> par les valeurs correspondantes).

Si vous avez des éléments supplémentaires à fournir à votre correcteur back, vous pouvez ajouter à votre rendu un fichier d'explication ou de précisions qui sera nommé :

S4IL_BACK_202606_<votreNom>_<votrePrenom>_README.txt

Note sur les livrables

Un mini projet de ce type constitue une somme de travail supplémentaire conséquente, par rapport à un devoir sur un créneau ou même une journée, pour vos correcteurs. De ce fait, votre projet doit se lancer sans autre manipulation que :

- l'import de votre projet back dans STS puis son lancement.
- le remplacement du dossier src d'un projet Angular vierge par celui extrait de votre zip puis le lancement de ce projet via un « ng serve ».

Les seules exceptions à cette règle qui seront acceptées seront les précisions apportées via les fichiers README.txt fournis et le correcteur sera seul juge de la pertinence ou non des justificatifs fournis. En cas de doutes, n'hésitez pas à poser des questions en amont (et pas la veille du rendu à 0h42!).

Listes des compétences évaluées

Comme d'habitude, nous vous précisons les compétences évaluées. **Prenez bien le temps de les lire et, pour chacune, posez-vous la question « Mon rendu couvre-t-il bien cette compétence ? »**

Nous avons trop souvent vu des rendus qui en faisaient plus que ce qui était demandé mais faisaient pourtant, par oubli en général, l'impasse sur une ou plusieurs compétences, pourtant listées dans ce paragraphe. Aller au rattrapage pour ce genre d'étourderie, ça peut être assez frustrant (pour le correcteur aussi).

Module 1 — Architecture 3-tiers Back-end (Spring Boot)

- | | | |
|---|---|--|
| 1 | Modéliser le domaine avec JPA | Concevoir des entités JPA avec annotations et relations (@Entity, @ManyToOne, @OneToMany, ...) |
| 2 | Implémenter l'architecture en couches | Controller -> Service -> repository avec injection de dépendances, DTOs et mapping. |
| 3 | Implémenter et documenter l'API REST | Réponses standardisées (ApiResponse), Swagger, ExceptionHandler. |
| 4 | Sécuriser l'API avec Spring Security et JWT | SecurityFilterChain, CORS, CSRF, sessions stateless, BCrypt ; génération/validation du token, JWTAuthenticationFilter, endpoints /register et /login. |
| 5 | Créer des tests unitaires avec JUnit 5 | Tester indépendamment les différentes couches (repository, service et controller). Utilisation de Mockito avec annotations de test (@DataJpaTest, @ExtendWith, @Mock, @InjectMocks, ...) |

Module 2 — Architecture 3-tiers Front-end (Angular)

- | | | |
|---|--|---|
| 1 | Créer et faire communiquer les composants, services et modèles d'une application Angular | Créer et utiliser un composant, créer et utiliser un service, créer et utiliser un modèle |
| 2 | Lier et synchroniser les données entre la vue et la logique applicative | Interpolation, property binding, event binding et signaux |
| 3 | Gérer la navigation et les formulaires d'une application Angular | Mettre en place des routes, créer un formulaire |
| 4 | Récupérer et traiter des données distantes via les requêtes HTTP et les observables | Effectuer des requêtes HTTP, utiliser des observables |

Module 3 — Intégration Full-Stack

- | | | |
|---|--|--|
| 1 | Consommer l'API sécurisée | Appeler l'API Spring Boot depuis Angular avec HttpClient. Envoyer et gérer le JWT côté client pour l'authentification. |
| 2 | Protéger les interfaces Angular | Restreindre l'accès aux routes selon l'état d'authentification avec les Guards Angular. Gérer la navigation conditionnelle selon les droits utilisateur. |
| 3 | Optimiser la gestion des données côté client | Centraliser et partager les données dans l'application via les services Angular. Gérer les flux asynchrones et la synchronisation avec le serveur grâce aux observables. |